

NN-MPC: Accelerating Model Predictive Control with Neural Networks for Fast Humanoid Locomotion Optimization

Abstract—Model Predictive Control (MPC) faces significant computational barriers in real-time humanoid locomotion, particularly on resource-constrained edge devices. This paper proposes NN-MPC, a neural network-accelerated framework that replaces MPC solvers with a LSTM network to approximate optimal control policies directly. By integrating temporal sequence modeling, stochastic data augmentation, and a hybrid stance-swing leg control architecture, the framework achieves 6- to 12-fold faster computation than state-of-the-art solvers (qpOASES/OSQP) while maintaining robust performance. Experimental validation on a 21-DoF humanoid robot and Unitree G1 demonstrates high success rate in locomotion tasks, generalization to untrained velocities, and stable disturbance rejection. Although limited in handling highly nonlinear dynamics and uneven terrains, NN-MPC bridges computational efficiency and complex locomotion, enabling practical deployment in logistics and assistive robotics. This work pioneers neural network-based optimization for large-scale humanoid systems, offering a viable path toward autonomous robots in dynamic environments.

Index Terms—Model predictive control, neural network-based optimization, humanoid locomotion

I. INTRODUCTION

Humanoid locomotion has advanced significantly, transitioning from niche prototypes to deployable systems. This progress has been driven by a combination of innovations in design and control, with optimization-based control strategies playing a central role in achieving reliable and efficient locomotion [1]. Current robots now demonstrate robust locomotion in nominal environments, paving the way for practical deployments in applications such as logistics, agriculture, and home assistance [2], [3], [4], [5].

Optimization-based control has been a cornerstone of these advancements, offering a structured approach to solving the complex challenges associated with humanoid locomotion. These approaches often utilize a whole-body MPC controller or a simplified model, such as centroidal dynamics, in conjunction with local task-space whole-body controllers (WBC) [1]. Typically, these control strategies are formulated as optimal control problems (OCPs) and are solved using standard or custom numerical solvers. However, the computational demands of these methods, particularly in resource-limited environments, pose a significant challenge.

MPC faces significant challenges in reducing computational costs due to the inherent complexity of numerical optimization. The humanoid locomotion optimization problem in MPC is often nonconvex, leading to high computational demands, sensitivity to initialization, and no guarantee of finding a solution or proving its absence [6]. These challenges are compounded by the need for real-time computation, as MPC

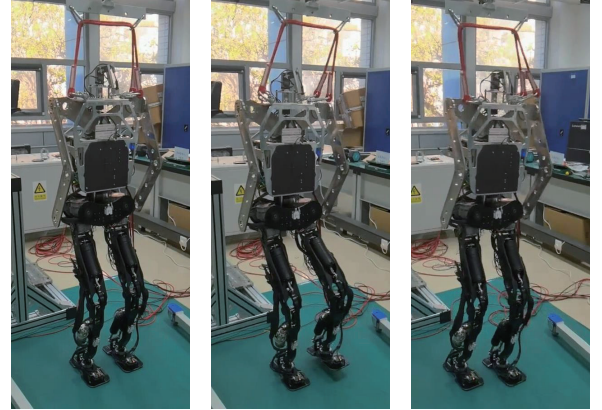


Fig. 1. Experimental platform: Physical humanoid robot with hybrid leg actuation.

must repeatedly solve the optimization problem for the next few time steps to adapt to evolving mission requirements.

To address this issue, there is a growing interest in enhancing OCPs by integrating machine learning (ML) techniques. Among these, a particularly effective approach involves replacing the optimization solver with a NN trained to imitate the solver. This method, known as neural network-based optimization (NNBO), neural solvers, or neural approximations [7], simplifies the optimization process into straightforward NN prediction. The key distinction between NNBO and other ML-augmented OCP methods, such as warm-start solvers or value function approximation, lies in its direct approximation of MPC. NNBO can be applied universally to all types of MPC, requires minimal storage space, and achieves a reduction in computation time by several orders of magnitude.

In this work, we propose a framework that leverages the unique strengths of NNBO for humanoid locomotion and validate its efficiency in high-dimensional scenarios through experiments (Fig. 1). Different signal-to-noise ratio (SNR) levels of white noise data are incorporated into the feedback loop as part of a data augmentation strategy. This approach has been shown to effectively mitigate overfitting and enhance robustness in our experiments while maintaining minimal performance loss. Under these conditions, the computational cost is reduced by 6- to 12-fold. Our contributions include:

- A framework that combines the strengths of *offline* learning and *online* optimization for efficient humanoid locomotion within MPC formulations.

TABLE I
A COMPREHENSIVE OVERVIEW OF METHODS FOR INTEGRATING ML WITH OCPs

Ref	Approach	OCPs' parameters				NN's parameters		Results	
		n_x	n_y	n_u	N	Architecture	Hidden layers	Speed up	Validation
Romualdi et al. [8]	Hierarchical control	55	31	26	24	MANN ¹	$512 \times 512 \times 512$	—	Humanoid robot
Banerjee et al. [9]	Warm-starting	26	19	19	—	FNN	$256 \times 512 \times 256$	—	Sim.
Celestini et al. [10]	Warm-starting	6	3	3	30	Transformer	—	7-fold	UAV
Dr̄goña et al. [11]	NNBO	286	6	6	22	Time delay NN	24×12	7.6-fold	Sim.
Nurbayeva [12]	NNBO	6	6	6	10	FNN	$46 \times 36 \times 27$	1.5-fold	UR5
Dr̄goña et al. [13]	NNBO	—	1	1	32	BS-SSM ²	$20 \times 20 \times 20$	1.3-fold	Laboratory device
Zhang et al. [14]	NNBO	4	3	3	3	FNN	—	10-fold	Automotive ECU
Wang et al. [15]	NNBO	3	3	2	50	FNN	$35 \times 35 \times 35 \times 35 \times 35$	6- to 72-fold	Sim.
Li et al. [16]	NNBO	4	2	2	20	RNN	20	1000-fold	Sim.
Ours	NNBO	48	12	12	10	LSTM	$512 \times 512 \times 512$	6- to 12-fold	Humanoid robot

* n_x means the number of states, n_y means the number of outputs of plants, n_y means the number of inputs of plants, N means the length in steps of the prediction or control horizons.

¹ Mode-adaptive neural network [17].

² Block-structured neural state-space model.

- Addressing the common issue of approximation inaccuracies in NNBO by introducing a series of regularization techniques to improve the robustness and stability of the NN-MPC controller.
- Demonstrating the effectiveness of the proposed framework through simulations and experiments on a real-world humanoid robot platform. Our results show significant improvements in MPC performance, including reductions in cost, runtime, and convergence rates, highlighting the framework's applicability to real-world scenarios with nonlinear dynamics and constraints.

II. RELATED WORK

Research on NNBO in humanoid locomotion remains limited. Relevant literature can be categorized into three areas: (i) ML-enhanced optimal control in humanoid locomotion, (ii) ML-enhanced optimal control in other domains, and (iii) direct NN-based approximations of optimization problems (NNBO).

A. ML for Optimal Control in Humanoid Locomotion

Recent works have explored ML to enhance locomotion robustness and stability through hierarchical or hybrid control architectures. Some methods use learning-based modules to generate gait trajectories followed by MPC tracking [8], or to complement MPC via genetic algorithms and reinforcement learning [18], [19]. Others incorporate learning within MPC itself, e.g., for contact modeling [20], to improve both fidelity and efficiency. Notably, due to real-time constraints, most humanoid MPC implementations rely on simplified models such as angular-momentum-based LIPM [18] or single rigid-body dynamics [8], [19], [20].

B. ML for Optimal Control in Other Domains

In aerospace and robotics, ML has been widely used to warm-start optimization problems and reduce solver iterations [9], [21]. Although effective, this strategy depends heavily on training data quality and may not guarantee constraint satisfaction. More recently, transformer architectures have been applied to optimal control due to their sequence modeling capabilities. For example, aerospace applications show

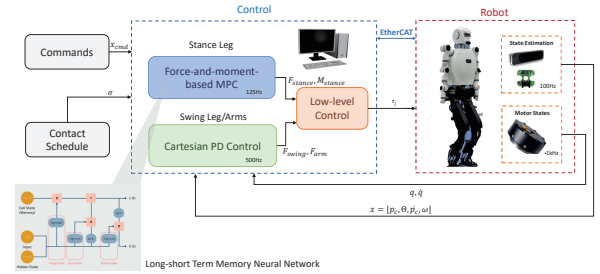


Fig. 2. **System and architecture of NN-MPC:** A LSTM network approximates the MPC to control stance leg.

improved convergence and runtime by integrating transformers for trajectory generation [10], [22]. However, such models require large datasets and have high inference latency, limiting their application in real-time millisecond-level control.

C. NNBO in Other Domains

Unlike modular integration of ML within MPC, NNBO seeks to directly approximate the control law of MPC using neural networks, enabling faster inference for real-time applications. Gonzalez et al. [7] categorize NNBO into two main types: (i) imitation-based regression using MSE or RMSE loss [11], [12], and (ii) enhanced approaches with MPC-inspired loss or primal-dual learning [13], [14], [15]. For example, Zhang et al. [14] proposed a primal-dual NNBO framework achieving 10-fold acceleration on automotive-grade hardware, while retaining constraint checks. Further extensions incorporate statistical or probabilistic validation [14], [16]. However, most studies target low-dimensional systems (few states/inputs) and are evaluated primarily in simulation [11], [23]. To date, no NNBO framework has demonstrated real-time control in high-dimensional humanoid systems exceeding 40 states, as presented in this work.

III. HUMANOID LOCOMOTION MPC

This section presents the baseline MPC framework for humanoid locomotion. Although whole-body dynamics can improve model fidelity, we adopt the simplified single rigid-body dynamics (SRBD) formulation that neglects the inertial

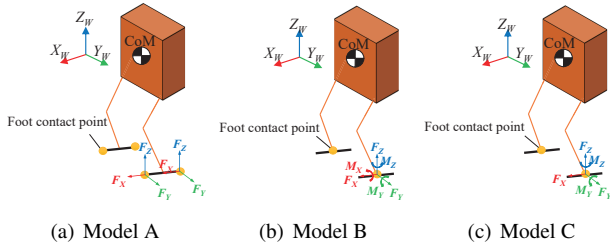


Fig. 3. **Contact Modeling Configurations for SRBD.** Schematic representations of ground interaction paradigms: (a) Dual contact points per foot with 3D force vectors; (b) Single contact point per foot with full 3D wrench; (c) Simplified contact model with planar moment constraints. Yellow markers indicate contact locations, while red arrows denote force/moment applications.

effects of leg links to ensure real-time computability of the MPC solution. As depicted in Fig. 2, the control architecture consists of four primary components: (i) a force-and-moment-based MPC for stance leg control, (ii) a Cartesian PD controller for swing leg trajectory tracking, (iii) a low-level joint controller, and (iv) a state estimation module.

A. Notation

- $\mathbf{I}_n$ and $\mathbf{0}_n$ denote the $n \times n$ identity and zero matrices, respectively;
- $\mathbf{g} = [0, 0, 9.81]^\top \in \mathbb{R}^3$ represents the gravitational acceleration vector;
- $\mathbf{p}_c \in \mathbb{R}^3$ denotes the position of robot's center of mass (CoM);
- $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ constitutes the continuous-time state-space representation, where the input vector $\mathbf{u} = [\mathbf{F}_1^\top, \mathbf{F}_2^\top, \mathbf{M}_1^\top, \mathbf{M}_2^\top]^\top \in \mathbb{R}^{12}$ encapsulates the ground reaction wrenches (GRWs) generated by the contact feet. Here $\mathbf{F}_i = [F_{i,x}, F_{i,y}, F_{i,z}]^\top$ and $\mathbf{M}_i = [M_{i,x}, M_{i,y}, M_{i,z}]^\top$ represent the three-dimensional contact force and moment vectors, respectively, with the subscript $i \in \{1, 2\}$ indexing the foot-end effector.

B. SRBD

Three predominant contact modeling methodologies are prevalent in legged robotic systems, as illustrated in Fig. 3. Model A extends the quadrupedal SRBD formulation [24] to bipedal systems through four contact points with three-dimensional force vectors. However, numerical simulations reveal significant pitch dynamics modeling errors during rotational maneuvers in humanoid applications [25]. Model B resolves this limitation by employing single-point ground contacts with full 3D wrench representation. As a constrained variant of Model B, Model C incorporates a planar moment approximation of the contact wrench cone (CWC) [26]. While effective for 10-DoF humanoid platforms with locked ankle roll joints [27], our hardware experiments demonstrate its inadequacy in capturing roll dynamics for 12-DoF lower-limb architectures. Consequently, the complete wrench formulation of Model B is implemented as our SRBD foundation.

The SRBD formulation governing the CoM translational acceleration $\ddot{\mathbf{p}}_c$ and body angular acceleration $\dot{\boldsymbol{\omega}}$ is expressed

through Newton-Euler equations:

$$m(\ddot{\mathbf{p}}_c + \mathbf{g}) = \sum_i^2 \mathbf{F}_i \quad (1)$$

$$\frac{d}{dt}(\mathcal{G}\mathbf{I}\boldsymbol{\omega}) = \sum_i^2 [\mathbf{p}_i - \mathbf{p}_c] \times + \mathbf{M}_i$$

where $m \in \mathbb{R}^+$ denotes total mass, $\mathcal{G}\mathbf{I} \in \mathbb{S}_{++}^3$ the centroidal inertia tensor, $\mathbf{p}_i \in \mathbb{R}^3$ the i -th foot position. The term $[\mathbf{p}_i - \mathbf{p}_c] \times$ is the skew-symmetric matrix representing the cross product of $(\mathbf{p}_i - \mathbf{p}_c) \times \mathbf{F}_i$.

The kinematic relationship between Euler angle derivatives $\dot{\boldsymbol{\Theta}} = [\dot{\phi}, \dot{\theta}, \dot{\psi}]^\top$ (roll, pitch, yaw), and the body angular velocity $\boldsymbol{\omega}$ is given by:

$$\boldsymbol{\omega} = \mathcal{G}\mathbf{R}(\boldsymbol{\Theta})\dot{\boldsymbol{\Theta}} = \begin{bmatrix} \sin \theta \sin \psi & \cos \psi & 0 \\ \sin \theta \cos \psi & -\sin \psi & 0 \\ \cos \theta & 0 & 1 \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} \quad (2)$$

where $\mathcal{G}\mathbf{R} \in SO(3)$ represents the rotation matrix from Euler angles to body-fixed coordinates.

C. MPC Formulation

Following the dynamic model derivation, we formalize the MPC formulation. Defining the state vector as $\mathbf{x} = [\boldsymbol{\Theta}^\top, \mathbf{p}_c^\top, \boldsymbol{\omega}^\top, \dot{\mathbf{p}}_c^\top]^\top \in \mathbb{R}^{12}$ and control input $\mathbf{u}$, the continuous-time dynamics in the world frame $\mathcal{G}$ are expressed as:

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{C}\mathbf{g} \quad (3)$$

To linearize the gravity term, we augment the state vector with a dummy gravity state $\tilde{\mathbf{g}}$, yielding $\tilde{\mathbf{x}} = [\mathbf{x}^\top, \tilde{\mathbf{g}}]^\top \in \mathbb{R}^{13}$:

$$\dot{\tilde{\mathbf{x}}} = \tilde{\mathbf{A}}\tilde{\mathbf{x}} + \tilde{\mathbf{B}}\mathbf{u} \quad (4)$$

$$\tilde{\mathbf{A}} = \begin{bmatrix} \mathbf{A} & \mathbf{C} \\ \mathbf{0}_{1 \times 12} & \mathbf{0} \end{bmatrix} \quad \tilde{\mathbf{B}} = \begin{bmatrix} \mathbf{B} \\ \mathbf{0}_{1 \times 12} \end{bmatrix}$$

Discretization using zero-order hold with sampling time Δt yields the discrete-time prediction model:

$$\tilde{\mathbf{x}}[k+1] = \mathbf{A}_d\tilde{\mathbf{x}}[k] + \mathbf{B}_d\mathbf{u}[k] \quad (5)$$

where $\mathbf{A}_d = \mathbf{I}_{13} + \tilde{\mathbf{A}}\Delta t$ and $\mathbf{B}_d = \tilde{\mathbf{B}}\Delta t$.

The MPC problem with prediction horizon N is formulated with the following constraints:

Friction Cone Constraints (linearized pyramid):

$$-\mu_{\text{eff}}F_{i,z} \leq F_{i,x/y} \leq \mu_{\text{eff}}F_{i,z}, \quad \mu_{\text{eff}} = \mu/\sqrt{2} \quad (6)$$

where μ is the contact friction coefficient of the ground.

Vertical Force Bounds:

$$0 \leq F_{i,z} \leq F_{\text{max}} \quad (7)$$

Contact Wrench Cone Constraints [26]:

$$\begin{aligned} -l_t F_{\mathcal{F},z} &\leq M_{\mathcal{F},y} \leq l_h F_{\mathcal{F},z} \\ -l_w F_{\mathcal{F},z} &\leq M_{\mathcal{F},x} \leq l_w F_{\mathcal{F},z} \end{aligned} \quad (8)$$

$$\begin{aligned} \text{Yaw plane:} \quad &\pm \mu_{\text{eff}} M_{\mathcal{F},y} + M_{\mathcal{F},z} \leq l_t F_{\mathcal{F},y} \mp \mu_{\text{eff}} l_t F_{\mathcal{F},z} \\ &\pm \mu_{\text{eff}} M_{\mathcal{F},y} - M_{\mathcal{F},z} \leq l_h F_{\mathcal{F},y} \mp \mu_{\text{eff}} l_h F_{\mathcal{F},z} \end{aligned} \quad (9)$$

$$\text{Roll plane:} \quad \pm \mu_{\text{eff}} M_{\mathcal{F},x} \pm M_{\mathcal{F},z} \leq l_w F_{\mathcal{F},x} \mp \mu_{\text{eff}} l_w F_{\mathcal{F},z}$$

where $F_{\mathcal{F}} = \mathcal{G}_{\mathcal{F}}^T R^T F$ and $M_{\mathcal{F}} = \mathcal{G}_{\mathcal{F}}^T R^T M$ represent foot-frame wrench. l_h and l_t denote the distances from the origin of the frame $\{\mathcal{F}\}$ to the heel and toe, respectively. l_w refers to the distance from the origin of $\{\mathcal{F}\}$ to either the medial or lateral side of the foot, both of which are of equal length to the robot. The symbols $\pm$ and $\mp$ indicate that all combinations of these symbols should be considered.

Joint Torque Limits (via static Jacobian):

$$\tau_{\min} \leq J_c^T u \leq \tau_{\max} \quad (10)$$

where J_c denotes the contact Jacobian

The complete MPC optimization problem becomes:

$$\begin{aligned} \min_{x, u} \quad & \sum_{k=0}^{N-1} \|\tilde{x}[k+1] - \tilde{x}^{\text{cmd}}[k+1]\|_Q^2 + \|u[k]\|_R^2 \\ \text{s.t.} \quad & \text{Discrete linear dynamics: (5)} \\ & \text{Constraints: (6) ~ (10)} \\ & \tilde{x}[0] = \tilde{x}_{\text{init}} \end{aligned} \quad (11)$$

with $Q \succ 0$ and $R \succ 0$ as positive definite weight matrices.

D. QP Formulation

The extended prediction horizon in our MPC framework generates high-dimensional matrices that necessitate substantial computational resources. Nevertheless, the convex optimization structure permits efficient solution through condensed Quadratic Programming (QP) [28].

The dynamics constraints (5) over the N -step horizon are expressed in stacked form:

$$\mathbf{X} = \mathbf{A}_{qp} x_0 + \mathbf{B}_{qp} \mathbf{U} \quad (12)$$

where $\mathbf{X}$ contains predicted states $x[i+1], x[i+2] \dots x[i+N]$ and $\mathbf{U}$ contains current state $u[i]$ and next $N-1$ horizons, $u[i+1], u[i+2] \dots u[i+N-1]$.

The QP problem is formulated as:

$$\begin{aligned} \min_{\mathbf{U}} \quad & \frac{1}{2} \mathbf{U}^T \mathbf{H} \mathbf{U} + \mathbf{f}^T \mathbf{U} \\ \text{where,} \quad & \mathbf{H} = 2 \left(\mathbf{B}_{qp}^T \mathbf{Q}_{\text{ext}} \mathbf{B}_{qp} + \mathbf{R}_{\text{ext}} \right) + \epsilon \mathbf{I}, \\ & \mathbf{f} = 2 \mathbf{B}_{qp}^T \mathbf{Q}_{\text{ext}} (\mathbf{A}_{qp} x_0 - \mathbf{X}^{\text{cmd}}) \end{aligned} \quad (13)$$

where $\mathbf{Q}_{\text{ext}} = \text{blkdiag}(Q, \dots, Q)$ and $\mathbf{R}_{\text{ext}} = \text{blkdiag}(R, \dots, R)$ are block-diagonal weight matrices. The regularization term $\epsilon > 0$ ensures strict convexity through $\mathbf{H} \succ 0$, guaranteeing numerical solvability.

E. Swing Leg Control

While the MPC framework generates optimal GRWs for stance leg control through the static mapping:

$$\tau_{\text{stance}} = J_c^T \mathcal{G}_{\mathcal{F}} R^T u \quad (14)$$

This formulation provides no direct swing leg motion control. To address this limitation, we implement a Cartesian-space PD controller for swing leg trajectory tracking. The desired foot position p_f^{des} is synthesized using Raibert's heuristic [29]:

$$p_f^{\text{des}} = p_c + \frac{\dot{p}_c^{\text{cmd}} \Delta t}{2} \quad (15)$$

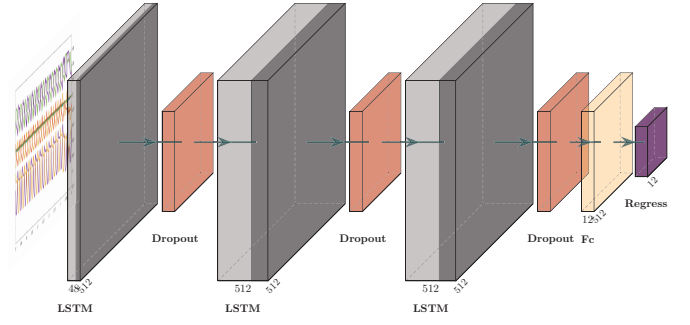


Fig. 4. LSTM network architecture for MPC policy approximation. The temporal sliding window input (curve figure) feeds into stacked LSTM layers (gray) with dropout regularization (dark red), producing time-distributed control outputs (yellow).

The desired joint angles q^{des} are obtained via inverse kinematics:

$$q^{\text{des}} = \text{IK}(p_f^{\text{des}}, \Phi^{\text{des}}) \quad (16)$$

with $\Phi^{\text{des}} \in \mathbb{R}^3$ specifying the desired foot orientation. The swing leg torque command combines position and velocity feedback:

$$\tau_{\text{swing}} = K_p (q^{\text{des}} - q) + K_d (\dot{q}^{\text{des}} - \dot{q}) \quad (17)$$

where $K_p, K_d \in \mathbb{S}_{++}$ are diagonal gain matrices.

The final torque command integrates both components:

$$\tau^{\text{cmd}} = \tau_{\text{stance}} + \tau_{\text{swing}} \quad (18)$$

The final torque control demand is summed τ_{stance} and τ_{swing} .

IV. NN-MPC IMPLEMENTATION

While the MPC framework is formulated as a QP problem, its computational complexity escalates significantly with extended prediction horizons. To address this challenge, we integrate a long short-term memory (LSTM) neural network into the control architecture to approximate the optimal MPC policy, as depicted in Fig. 2. This section details the training dataset generation methodology and presents key techniques for mitigating over-fitting in the learning-based controller.

A. LSTM-MPC Architecture Design

The proposed methodology adopts behavior cloning (BC) principles from imitation learning [30], where the control policy π_{θ} is trained through supervised learning on expert demonstration trajectories. The learning objective is formalized as:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(s,a) \sim \mathcal{B}} [\mathcal{L}(\pi_{\theta}(s), a)] \quad (19)$$

where $\mathcal{B}$ denotes the expert dataset containing MPC solution trajectories, and $\mathcal{L}$ represents the RMSE between neural network outputs and MPC reference commands:

$$\mathcal{L}(\tau) = \sum_{n=1}^{N_D} \sum_{t=1}^{N_h} \left(\|u_t^{(n)} - u_{t,MPC}^{(n)}\|_2^2 \right) \quad (20)$$

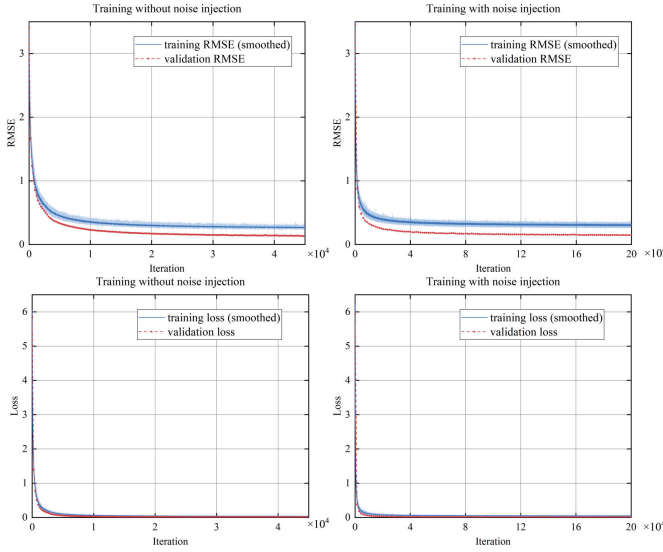


Fig. 5. Training performance comparison: (left) baseline without noise injection, (right) proposed method with stochastic augmentation

with $N_D = 12$ specifying the control input dimension and N_h defining the temporal sequence length.

A significant limitation of BC manifests in data-scarce scenarios [31]. The learned policy π_θ performs reliably only within the training distribution denoted by $\mathcal{B}$. For humanoid locomotion – a sequential decision-making problem with continuous state-action spaces – minor deviations from demonstrated trajectories can lead to unseen states during deployment. This distribution shift induces error accumulation through the recursive relationship:

$$s_{t+1} = f(s_t, \pi_\theta(s_t)) \quad (21)$$

where $f : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ represents the system dynamics model governing state transitions. To mitigate this compounding error problem, we employ temporal sequence modeling with sliding window inputs $\tau_t = \{s_{t-N_h+1}, \dots, s_t\}$ that preserve historical state correlations.

LSTM is chosen for its temporal modeling ability and efficiency on small datasets [32]. As shown in Fig. 4, the network comprises:

- * Input layer: Temporal sequences $\tau_t \in \mathbb{R}^{N_D \times N_h}$
- * Three LSTM layers: 512 hidden units per layer with tanh activation
- * Regularization: 20% dropout between LSTM layers
- * Output layer: Time-distributed dense layer with linear activation
- Weight regularization: L2 penalty with $\lambda = 10^{-4}$
- Early stopping: Training terminates if validation loss plateaus for 10 epochs
- Input normalization: Online z-score standardization

During deployment, only the final timestep prediction $u_t = \pi_\theta(\tau_t)[-1]$ is executed, maintaining causal consistency in closed-loop control.

B. Stochastic Training Framework

The acquisition of MPC training data through physical sensor measurements presents prohibitive temporal demands, typically requiring weeks of continuous operation. Furthermore, iterative parameter tuning of the MPC formulation necessitates repeated data collection cycles. To address these challenges, Section V-A introduces a physics-based simulator that accelerates trajectory generation by three orders of magnitude compared to real-world data acquisition

To mitigate overfitting in the learning-based controller, we implement a structured data augmentation protocol. White noise injection with calibrated signal-to-noise ratios (SNRs) enhances model robustness against sensor uncertainties. The augmentation process comprises:

- * Signal power analysis for measured states:

$$P_{\text{signal}} = \mathbb{E}[|x|^2], \quad x \in q, p_i, \dot{p}_i \quad (22)$$

- * Noise injection with SNR control:

$$A_{\text{noise}} \sim \mathcal{N}(0, \sigma^2), \quad \sigma^2 = P_{\text{signal}} \cdot 10^{-\text{SNR}/10} \quad (23)$$

- * Stochastic augmentation spanning 60-80 dB SNR at 5 dB intervals

The training configuration employs an 80-20% data partition for training-validation splitting, with gradient clipping threshold $\gamma = 1$ to prevent exploding gradients. As evidenced in Fig. 5, the proposed stochastic augmentation achieves fast convergence (353 early-stopping triggers at 160-minute runtime), demonstrating improved generalization capability.

V. VALIDATION AND EXPERIMENTS

This section presents comparative evaluations between the MPC framework and NN-MPC. The experimental protocol comprises three components: robotic platform specifications, simulation environment configuration, and benchmark scenarios for simulated and physical validation.

A. Robot Description

The humanoid robot features a series-parallel hybrid leg architecture distinguishing it from conventional revolute-joint humanoids. Key mechanical specifications include:

- * Anthropomorphic dimensions: Height = 1.73 m, Mass = 64.3 kg
- * Actuation system: 21 total degrees of freedom (DoFs)
 - Lower limbs: 6 DoFs per leg (3 hip, 1 knee, 2 ankle)
 - Upper body: 2 head, 2 waist, and 7 per arm DoFs
- * Sensing suite:
 - Xsens MTi-710 IMU (3D orientation @ 400 Hz)
 - 6-axis force-torque sensors per foot
 - Joint encoders (17-bit resolution)

The simulation environment, developed in MATLAB/Simulink R2024a with Simscape Multibody, addresses contact dynamics through equivalent constraint forces. OCPs are solved by CasADi 3.6.7 [33] with quadratic programming solvers qpOASES 3.2.1 [34] and OSQP 0.6.2 [35], separately.

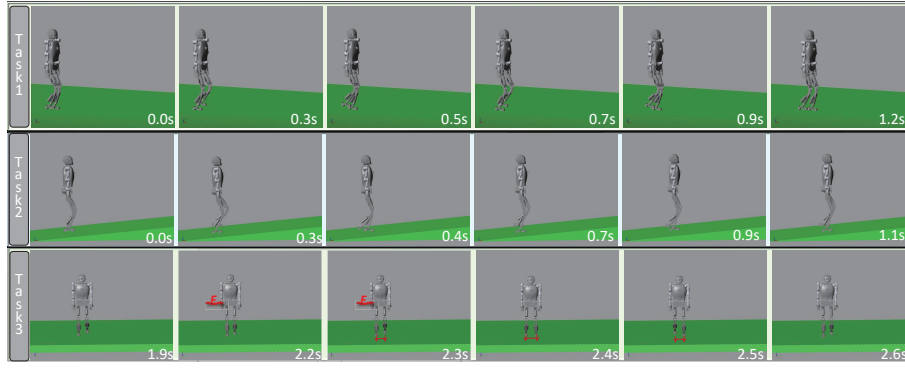


Fig. 6. Simulation results of three benchmark scenarios: **Task 1**: Level-ground walking at 1 m/s without disturbances, **Task 2**: Slope ascent (5°) at 0.7 m/s, **Task 3**: Lateral push recovery (50N impulse during 2.0-2.3s).

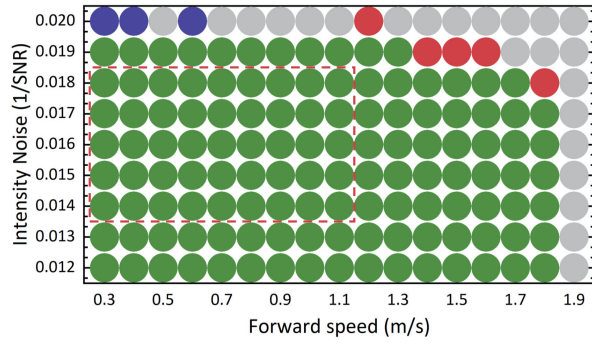


Fig. 7. Success rate as a function of velocity and noise intensity for NN-MPC, demonstrating generalization beyond training conditions (outside the red dashed rectangular box). Shaded regions indicate $\pm 1.5\sigma$ confidence intervals based on 50 trials. Green circles indicate successful execution by both NN-MPC and MPC; blue circles indicate success by NN-MPC only; red circles indicate success by MPC only; and grey circles indicate failure by both. The red dashed box represents the training dataset region.

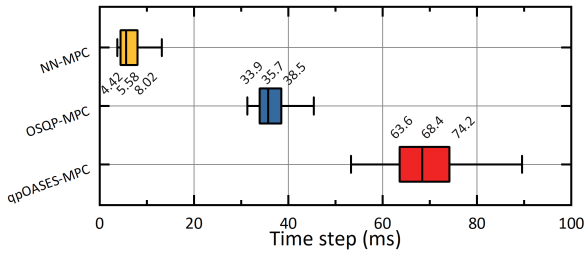


Fig. 8. Computational latency comparison: NN-MPC reduces mean step time to 5.58 ms (12.3-fold faster than qpOASES-MPC, 6.4-fold faster than OSQP-MPC). Error bars show 95th percentiles.

B. Case study

Three tasks—flat walking, slope ascent, and lateral push recovery—were used to validate robustness.

Fig. 6 demonstrates NN-MPC’s tracking performance under these scenarios. The success rate analysis in Fig. 7 reveals a trend: NN-MPC achieves 82.3% success rate within the training range and maintains robustness at extrapolation boundaries, where MPC fails. Success criteria require continuous walking for 5s with CoM projection inside support polygons.

The computational advantage of NN-MPC is quantified in Fig. 8. While traditional MPC solvers exhibit latency growth with problem complexity (qpOASES: 68.4 ± 4.8 ms, OSQP: 35.7 ± 1.8 ms), NN-MPC maintains consistent inference time (5.58 ± 1.16 ms) through parallelized network evaluation. This deterministic timing profile enables reliable higher control frequency.

C. Offline Hardware Experiment

Due to the computational limitations of the Speedgoat real-time controller, neither the MPC nor the NN-MPC was executed in a fully real-time fashion. Instead, joint position trajectories obtained from offline simulations were tracked by the robot through a PD controller. Despite neglecting actuator dynamics and surface irregularities, the NN-MPC-based reference trajectories achieved a tracking accuracy of 98.4% at a walking speed of 0.1 m/s.

D. Online Hardware Experiment

To validate the proposed NN-MPC method, we conducted experiments on the Unitree G1 humanoid robot. Leveraging the OCS2-based whole-body dynamics MPC framework [36], we accelerated deployment from RViz-based simulation to MuJoCo physics simulation and ultimately to real hardware, without fine-tuning the NN-MPC controller beyond the training setup described in Section IV.

As shown in Fig.9, the robot successfully performed walking tasks over diverse terrains—including grass, uneven paths, and 12 cm obstacles—despite being trained only on flat-ground walking in MuJoCo. This demonstrates the strong generalization capability and robustness of the NN-MPC framework in untrained real-world scenarios.

Fig.10 presents experimental results for humanoid walking. The left leg joint torques (Fig.10(b)) closely track the reference commands with clear periodicity. Minor deviations during foot contact remain within acceptable bounds, indicating accurate and stable control.

Table II compares the average computation time of a single optimization step for MPC and a forward pass of NN-MPC. The NN-MPC achieves a significant speedup, with a mean latency of 0.779 ms ($\sigma = 0.069$ ms), compared to 8.37 ms (with higher variance) for traditional MPC—yielding a $10.74\times$

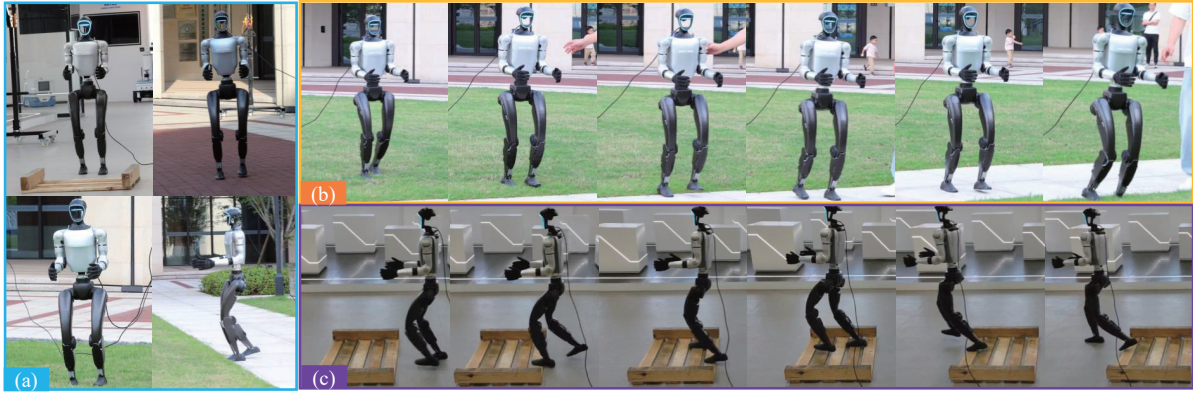


Fig. 9. Hardware walking experiments on diverse terrains. (a) Walking across four types of terrain. (b) Traversing grass and uneven path terrains. (c) Stepping over a 12 cm high wooden board.

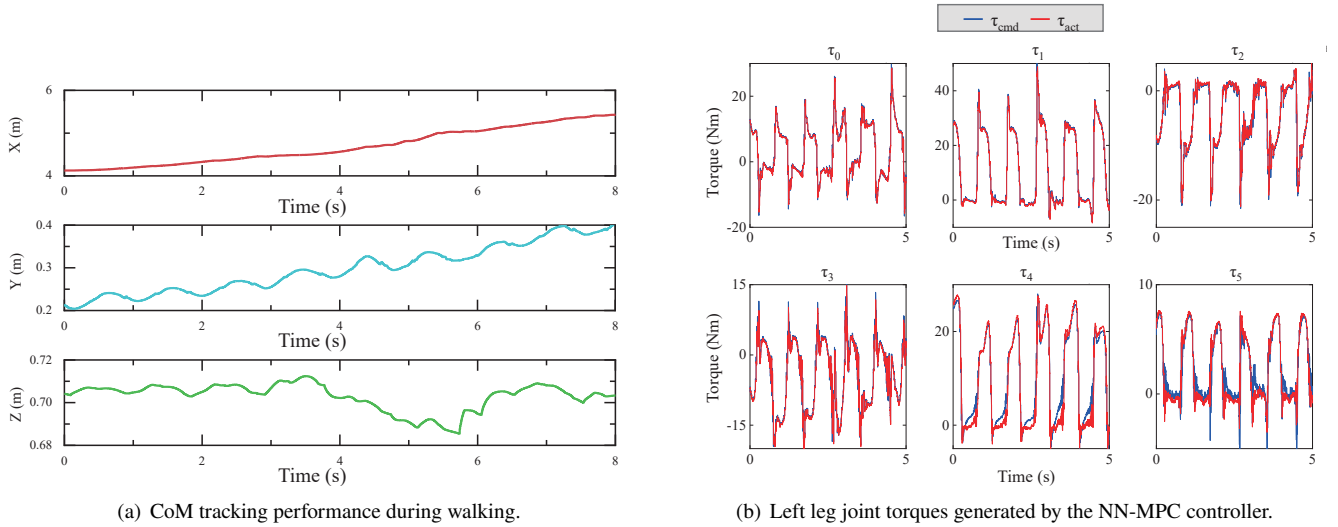


Fig. 10. Quantitative evaluation of the proposed NN-MPC framework for humanoid walking on flat ground. The CoM trajectory tracking and joint torque profiles illustrate the system’s ability to maintain dynamic balance while generating feasible control inputs.

TABLE II

COMPARISON OF COMPUTATION TIME BETWEEN SINGLE MPC OPTIMIZATION AND SINGLE FORWARD INFERENCE OF NN-MPC

Method	Average Time (ms)	Standard Deviation (ms)	Performance Gain
MPC	8.37	0.935	10.74-fold
NN-MPC	0.779	0.069	

improvement. This efficiency makes NN-MPC well suited for real-time, high-frequency humanoid control.

These results confirm the NN-MPC framework’s effectiveness in both control accuracy and computational efficiency, validating its practical applicability to real humanoid systems.

VI. CONCLUSIONS

This paper presented NN-MPC, a neural network-accelerated framework for real-time humanoid locomotion optimization. By learning a direct mapping from system states to control actions using an LSTM-based approximation of

the MPC policy, the proposed method achieves a 6- to 12-fold reduction in computational latency compared to state-of-the-art solvers like qpOASES and OSQP, while maintaining minimal performance degradation. Key innovations include the integration of stochastic data augmentation and temporal sequence modeling to mitigate approximation inaccuracies and distribution shifts, ensuring robust control under noisy and dynamic conditions. Experimental validation on both simulation and physical platforms demonstrated the framework’s ability to generalize beyond training conditions, achieving 82.3% success rate in locomotion tasks and stable tracking under disturbances.

Despite these advancements, the current framework faces limitations in handling highly nonlinear dynamics and out-of-distribution terrains, such as uneven surfaces or multi-contact scenarios. Future work will focus on extending NN-MPC to multi-task stochastic optimization through meta-learning and uncertainty-aware training strategies. Additionally, integrating reinforcement learning for adaptive fine-tuning and exploring hardware-aware neural architectures for edge deployment

could further enhance real-world applicability. By bridging the efficiency of neural networks with the robustness of optimization-based control, this work paves the way for deploying complex MPC formulations in resource-constrained robotic systems, with potential applications in disaster response, precision agriculture, and assistive technologies.

REFERENCES

- [1] P. M. Wensing, M. Posa, Y. Hu, A. Escande, N. Mansard, and A. D. Prete, "Optimization-based control for dynamic legged robots," *IEEE Transactions on Robotics*, vol. 40, pp. 43–63, 2024.
- [2] M. Posa, C. Cantu, and R. Tedrake, "A direct method for trajectory optimization of rigid bodies through contact," *The International Journal of Robotics Research*, vol. 33, no. 1, pp. 69–81, 2014.
- [3] J. Vaillant, A. Kheddar, H. Audren, F. Keith, S. Brossette, A. Escande, K. Bouyarmane, K. Kaneko, M. Morisawa, P. Gergondet, et al., "Multi-contact vertical ladder climbing with an hrp-2 humanoid," *Autonomous Robots*, vol. 40, no. 3, pp. 561–580, 2016.
- [4] A. Hereid, E. A. Cousineau, C. M. Hubicki, and A. D. Ames, "3d dynamic walking with underactuated humanoid robots: A direct collocation framework for optimizing hybrid zero dynamics," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2016, pp. 1447–1454.
- [5] A. Kheddar, S. Caron, P. Gergondet, A. Comport, A. Tanguy, C. Ott, B. Henze, G. Mesesan, J. Engelsberger, M. A. Roa, et al., "Humanoid robots in aircraft manufacturing: The airbus use cases," *IEEE Robotics & Automation Magazine*, vol. 26, no. 4, pp. 30–45, 2019.
- [6] S. Boyd, "Convex optimization," *Cambridge UP*, 2004.
- [7] C. Gonzalez, H. Asadi, L. Kooijman, and C. P. Lim, "Neural networks for fast optimisation in model predictive control: a review," *arXiv preprint arXiv:2309.02668*, 2023.
- [8] G. Romualdi, P. M. Viceconte, L. Moretti, I. Sorrentino, S. Daffarra, S. Traversaro, and D. Pucci, "Online dnn-driven nonlinear mpc for stylistic humanoid robot walking with step adjustment," in *2024 IEEE-RAS 23rd International Conference on Humanoid Robots (Humanoids)*. IEEE, 2024, pp. 584–591.
- [9] S. Banerjee, T. Lew, R. Bonalli, A. Alfaadhel, I. A. Alomar, H. M. Shageer, and M. Pavone, "Learning-based warm-starting for fast sequential convex programming and trajectory optimization," in *2020 IEEE Aerospace Conference*. IEEE, 2020, pp. 1–8.
- [10] D. Celestini, D. Gammelli, T. Guffanti, S. D'Amico, E. Capello, and M. Pavone, "Transformer-based model predictive control: Trajectory optimization via sequence modeling," *IEEE Robotics and Automation Letters*, 2024.
- [11] J. Drgoňa, D. Picard, M. Kvasnica, and L. Helsen, "Approximate model predictive building control via machine learning," *Applied Energy*, vol. 218, pp. 199–216, 2018.
- [12] A. Nurbayeva, A. Shintemirov, and M. Rubagotti, "Deep imitation learning of nonlinear model predictive control laws for a safe physical human-robot interaction," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 7, pp. 8384–8395, 2022.
- [13] J. Drgoňa, K. Kiš, A. Tuor, D. Vrabie, and M. Klaučo, "Differentiable predictive control: Deep learning alternative to explicit model predictive control for unknown nonlinear systems," *Journal of Process Control*, vol. 116, pp. 80–92, 2022.
- [14] X. Zhang, M. Bujarbaruah, and F. Borrelli, "Near-optimal rapid mpc using neural networks: A primal-dual policy learning framework," *IEEE Transactions on Control Systems Technology*, vol. 29, no. 5, pp. 2102–2114, 2020.
- [15] R. Wang, H. Li, and D. Xu, "Learning model predictive control law for nonlinear systems," in *2022 5th International Symposium on Autonomous Systems (ISAS)*. IEEE, 2022, pp. 1–6.
- [16] Y. Li, K. Hua, and Y. Cao, "Using stochastic programming to train neural network approximation of nonlinear mpc laws," *Automatica*, vol. 146, p. 110665, 2022.
- [17] H. Zhang, S. Starke, T. Komura, and J. Saito, "Mode-adaptive neural networks for quadruped motion control," *ACM Transactions on Graphics (TOG)*, vol. 37, no. 4, pp. 1–11, 2018.
- [18] S. H. Bang, C. A. Jové, and L. Sentis, "RL-augmented mpc framework for agile and robust bipedal footstep locomotion planning and control," in *2024 IEEE-RAS 23rd International Conference on Humanoid Robots (Humanoids)*. IEEE, 2024, pp. 607–614.
- [19] M. Kasaei, M. Abreu, N. Lau, A. Pereira, and L. P. Reis, "Robust biped locomotion using deep reinforcement learning on top of an analytical control approach," *Robotics and Autonomous Systems*, vol. 146, p. 103900, 2021.
- [20] Q. Chen, J. Li, S. Cheng, N. Hovakimyan, and Q. Nguyen, "Autotuning bipedal locomotion mpc with grfm-net for efficient sim-to-real transfer," *arXiv preprint arXiv:2409.15710*, 2024.
- [21] J. Briden, C. Choi, K. Yun, R. Linares, and A. Cauligi, "Constraint-informed learning for warm starting trajectory optimization," *arXiv preprint arXiv:2312.14336*, 2023.
- [22] T. Guffanti, D. Gammelli, S. D'Amico, and M. Pavone, "Transformers for trajectory optimization with application to spacecraft rendezvous," in *2024 IEEE Aerospace Conference*. IEEE, 2024, pp. 1–13.
- [23] S. W. Chen, T. Wang, N. Atanasov, V. Kumar, and M. Morari, "Large scale model predictive control with neural networks and primal active sets," *Automatica*, vol. 135, p. 109947, 2022.
- [24] G. Bledt, M. J. Powell, B. Katz, J. Di Carlo, P. M. Wensing, and S. Kim, "Mit cheetah 3: Design and control of a robust, dynamic quadruped robot," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 2245–2252.
- [25] J. Li and Q. Nguyen, "Force-and-moment-based model predictive control for achieving highly dynamic locomotion on bipedal robots," in *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, 2021, pp. 1024–1030.
- [26] S. Caron, Q.-C. Pham, and Y. Nakamura, "Stability of surface contacts for humanoid robots: Closed-form formulae of the contact wrench cone for rectangular support areas," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2015, pp. 5107–5112.
- [27] Y. Ding, C. Khazoom, M. Chignoli, and S. Kim, "Orientation-aware model predictive control with footstep adaptation for dynamic humanoid walking," in *2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*. IEEE, 2022, pp. 299–305.
- [28] J. L. Jerez, E. C. Kerrigan, and G. A. Constantinides, "A condensed and sparse qp formulation for predictive control," in *2011 50th IEEE Conference on Decision and Control and European Control Conference*. IEEE, 2011, pp. 5217–5222.
- [29] M. H. Raibert, *Legged robots that balance*. MIT press, 1986.
- [30] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.
- [31] F. Codevilla, M. Müller, A. López, V. Koltun, and A. Dosovitskiy, "End-to-end driving via conditional imitation learning," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 4693–4700.
- [32] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [33] J. A. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "Casadi: a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, pp. 1–36, 2019.
- [34] H. J. Ferreau, C. Kirches, A. Potschka, H. G. Bock, and M. Diehl, "qpocases: A parametric active-set algorithm for quadratic programming," *Mathematical Programming Computation*, vol. 6, pp. 327–363, 2014.
- [35] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "Osqp: An operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.
- [36] M. Y. Galliker, "Whole-body humanoid mpc: Realtime physics-based procedural loco-manipulation planning and control," https://github.com/1x-technologies/wb_humanoid_mpc, 2024.